

Task Interruption in Software Development Projects

What Makes some Interruptions More Disruptive than Others?

Zahra Shakeri Hossein Abad*, Oliver Karras†, Kurt Schneider‡, Ken Barker*, Mike Bauer‡

* Department of Computer Science, University of Calgary, Canada, {zshakeri, kbarker}@ucalgary.ca

† Software Engineering Group, Leibniz Universität Hannover, Germany, {oliver.karras, kurt.schneider}@inf.uni-hannover.de

‡ Arcurve Inc., Calgary, Canada, mike.bauer@arcurve.com

ABSTRACT

Multitasking has always been an inherent part of software development and is known as the primary source of interruptions due to task switching in software development teams. Developing software involves a mix of analytical and creative work, and requires a significant load on brain functions, such as working memory and decision making. Thus, task switching in the context of software development imposes a cognitive load that causes software developers to lose focus and concentration while working thereby taking a toll on productivity. To investigate the disruptiveness of task switching and interruptions in software development projects, and to understand the reasons for and perceptions of the disruptiveness of task switching we used a mixed-methods approach including a longitudinal data analysis on 4,910 recorded tasks of 17 professional software developers, and a survey of 132 software developers. We found that, compared to task-specific factors (e.g. priority, level, and temporal stage), contextual factors such as interruption type (e.g. self/external), time of day, and task type and context are a more potent determinant of task switching disruptiveness in software development tasks. Furthermore, while most survey respondents believe external interruptions are more disruptive than self-interruptions, the results of our retrospective analysis reveals otherwise. We found that self-interruptions (i.e. voluntary task switchings) are more disruptive than external interruptions and have a negative effect on the performance of the interrupted tasks. Finally, we use the results of both studies to provide a set of comparative vulnerability and interaction patterns which can be used as a mean to guide decision-making and forecasting the consequences of task switching in software development teams.

KEYWORDS

Multitasking, Task switching, Task interruption, Productivity, Retrospective analysis, Empirical software engineering

1 INTRODUCTION

Software development has undergone significant changes over the past decade. Traditionally siloed development teams are more collaborative and included more stakeholders from more disciplines than ever before. The need for faster-time-to-market, frequent releases, continuous integration, and continuous delivery has made

frequent task switching an unavoidable part of software development projects. *Task switching*, commonly referred to as *multitasking* [36] and *interruption* [30] is the act of starting one task and moving to another before finishing the first. Developers often have to switch tasks for various reasons: getting sidetracked to other tasks; getting stuck or bored by complex or lengthy repetitive tasks; receiving priority change requests from the management team; or even something as simple as a question from a co-worker. In a recent study of interruptions Parnin and Rugaber [28] analyzed development logs of 10,000 programming sessions from 86 programmers and found that in a typical day, a developer's work is fragmented into many short sessions (i.e. 15-30 minutes), and a programmer often spends a significant amount of time (i.e. 15-30 minutes) reconstructing working context before resuming interrupted tasks. To gain a better grasp of the behaviour of task switching in software development projects we conducted an investigation of 44,515 tasks (recorded between 2013 and 2017) of 23 professional software developers at SET GmbH¹, a leading provider of standard software for output management². We found that developers switch about two-thirds (59%) of their daily tasks from which 40% require context switching, and they never resume 29% of their interrupted/switched tasks. While task switching in some cases help developers be more productive, it imposes a cognitive load on them: frequent task switching typically results in severe performance costs by increasing response latencies and error rates [15, 36], and can cause an initial decrease in how quickly people perform post-switching tasks [21].

Research into developers' productivity and multitasking provide evidence on how multitasking and interruptions can impact productivity in software development teams [22, 24, 36]. However, very little work [27, 28] has investigated the factors that can make task switching more disruptive for different types of software development tasks (e.g. programming, test, architecture, UI, and deployment). Given the paucity of empirical studies on the disruptiveness of task switching and interruption in software development projects, it remains unclear what factors make which types of task interruptions more disruptive than others. This paper reports on a mixed-methods study exploring and analyzing factors influencing the vulnerability of various types of software development tasks to interruptions. A multivariate longitudinal analysis was conducted to investigate disruptive factors, such as the interruption type (i.e. self/external), context switching, and interruption timing (i.e. daytime, task stage), and to then perform comparative

EASE'18-22nd International Conference on Evaluation and Assessment in Software Engineering 2018, Christchurch, New Zealand

2018. TBA.

DOI: TBA

¹<https://www.set.de>

²This analysis has been conducted to justify our research goals and it is different from the main longitudinal study of this paper

and cross-factor analysis on the vulnerability of various software development tasks based on these factors. Further, a survey of 132 professional software developers from different organizations (e.g. Microsoft, Tableau Software, Ericsson, Bosch, and Cisco) explored practitioners' perceptions of and reasons for task switching and the disruptiveness of these interruptions. These studies show that context switching (e.g. task type and project), the abstraction level (i.e. main task, sub-task) and the temporal stage (i.e. early, late) of the interrupted task, and the interruption type (i.e. self, external) significantly impact the disruptiveness of interruptions and task switching in software development tasks. In summary, this paper makes the following contributions:

- It models interruption characteristics and presents a longitudinal analysis of 4,910 task logs of 17 professional software developers to study the vulnerability of various development tasks to interruptions and to explore the disruptive impact of interruption characteristics on different tasks' types.
- It presents a survey of 132 professional software developers to identify their perceptions of the concept and impact of task switching and interruptions in software development projects.
- It provides a set of comparative disruptiveness as well as cross-factor interaction patterns that can be used to guide task switching and to predict and manage the cognitive load associated with various interruptions.

2 BACKGROUND

This section first describes concepts related to task switching and interruption. We formulate the dependent and independent variables of this study and conclude this section by reviewing the related work on interruption analysis in software engineering.

2.1 Terms and Concepts

The information required to accomplish a task decays gradually in human memory, which results in a mental clutter of goals/tasks. **Problem state**, the main source of interference in multitasking environments, keeps track of task-related information that is not readily available in the external environment [30] or in the information associated with performing a task. Some tasks are reactive (e.g. answering an email or phone calls) and do not need to maintain a problem state. Some tasks may utilize the problem state resource but do not need to maintain the information therein (e.g. stand-up meetings). As interference only arises when the problem state resource is needed by two or more tasks, tasks that do not require problem state information will not experience interference on the problem state resource. Thus, we do not consider switching from (or interrupting) reactive tasks and tasks that do not need to maintain their problem state as *task interruption*. Instead, we refer to this type of task switching as *no-task interruptions*. **Activation** (Λ) or the momentary availability of the memory content controls the speed and reliability of access to the memory content after resuming a task [7]. As activation grows the information can be retrieved in a shorter amount of time [30]. The time course of tasks activation in a sequential multitasking set-up is illustrated in Figure 1. The abscissa represents the time and the ordinate represents the activation level. The dashed line represents the **Interference level**

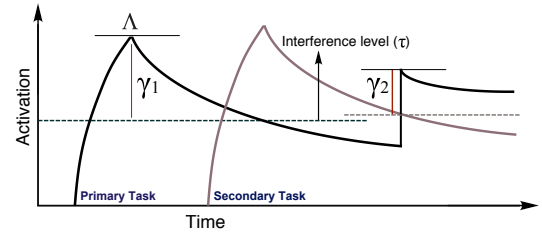


Figure 1: Goal activations in nested interruptions [adapted from Memory of Goals theory]. When $\gamma = 0$ the probability of recall is %50.

(τ) (or activation threshold) and refers to the expected (mean) activation of the *most* interrupting task [6]. **Activation distance** (γ) represents the accuracy of memory for the current task and refers to the amount by which the resumed task at its peak is more active than the interference level. The memory-of-goals theory [6] shows that the interference level depends on the number of interrupting tasks (nested interruptions) and the long-term durability (i.e. strength) of the information associated with these tasks. The more they are, or the stronger they are, the more they interfere with the target [6], which contributes to a decrease in memory accuracy. For example, as illustrated in Figure 1, the memory accuracy decreases as the number of interrupting tasks increases ($\gamma_2 < \gamma_1$). The ACT-R theory computes the activation as a function of frequency of use (i.e. $\Lambda = \ln(\frac{n}{\sqrt{T}})$), where n is the total number of times the memory item has been retrieved in its lifetime, and T is the length of this lifetime. ACT-R formulates the probability of recall as an exponential function of activation distance (i.e. $P_{recall} = \frac{1}{1+e^{-\gamma/s}}$). Thus, as time passes without using an item, T for that item grows, whereas n does not, producing decay (a decrease in the activation level). Given that activation (Λ) decreases by time and the activation threshold (τ) (or the interference level) increases by the length of the interruptions and the number of distractors, we can conclude that the probability of recall decreases as a power function of *time* and *the number of distractors* [8]. Thus, in this paper, we study the vulnerability of software development tasks by exploring the impact of various interruption characteristics on these two dependent variables: (1) **suspension length** [Δ], and (2) **the number of nested interruptions** [w]. Figure 2 presents the eight independent (ν_{1-8}) variables of this study, the way we interpreted them in the course of our data analysis, their data collection method, as well as their corresponding literature references.

2.2 Related Work

Characterizing, managing, and theorizing multitasking and task switching have received increasing research attention from different disciplines such as psychology [6, 30, 31], human-computer interaction [20, 21, 29], and management [26]. In addition to the related work discussed in Section 2.1, we focus on research related to multitasking and interruptions in the area of software engineering. Looking at multitasking and productivity, Vasilescu *et al.* [36] developed models and methods to measure the rate and breadth of developer's context-switching behaviour and studied how the switching behaviour affects developers' productivity. They found that a high rate of project switching per day results in a

▲ (Primary Task's Context, [Project Id], [Company X's Dataset*])	◆ (Priority Change, [Same, Different], [Data Analysis])
▲ (Secondary Task's Context, [Project Id], [Company X's Dataset])	▲ (Experience Level, [Avg. 12.5], [Company X's Dataset, LinkedIn***])
◆ (Primary Task's Type, [Arch, Prog, Test, UI, Dep], [Manual Data Analysis**])	● (Task Level, [Sub-task, Main], [Data Analysis])
◆ (Secondary Task's Type, [Arch, Prog, Test, UI, Dep], [Manual Data Analysis])	● (Task's Temporal Stage, [Early, Late], [Manual Data Analysis])
■ (Interruption Type, [Self, External], [Manual Data Analysis])	○ (Suspension Period, [0 – 10 days], Manual Data Analysis)
⌚ (Daytime, [Morning, Afternoon], [Data Analysis])	⊗ (Nested Interruption, [1 – 15 tasks], Manual Data Analysis)

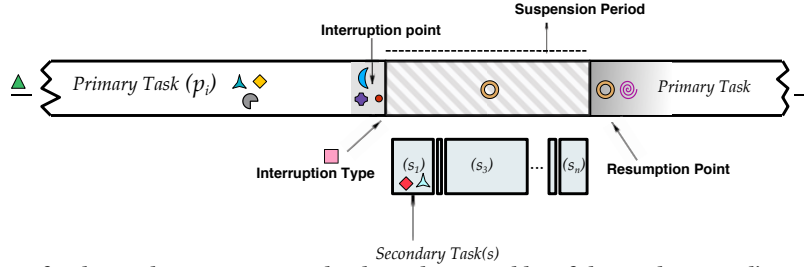


Figure 2: An overview of task switching spectrum and independent variables of this study. Legend's symbols can be interpreted as ⟨independent variable, [potential values], data collection method⟩. [*]: We used the project ID provided in the dataset to distinguish different contexts. []: We used text mining and manual analysis on the metadata associated with each task to explore the type of the tasks under study. [***]: We used employee's LinkedIn account to extract required information about their work experience. © Shakeri H.A , Karras, Schneider, Barker.**

Independent Variables (Task Switching Characteristics)

- ① **Context Switching [CS=1, Different project]** (iv_1): switching the project along with task switching [24, 36].
- ② **Type Difference [TD=1, Different type]** (iv_2): the type of the primary and the secondary tasks [10].
- ③ **Interruption Type [IT=1, Self]** (iv_3): **Self-interruption** if the interruption initiated by the subject of the primary task; **external-interruption** if it is motivated by some external events in the environment [30, 31].
- ④ **Daytime [DT=1, Morning]** (iv_4): The time of the day that task switching occurs [20]. All task switching and interruptions that were occurred between 11 am-1:30 pm (i.e. lunch time) were excluded from our analysis.
- ⑤ **Priority Change [PC=1, Same Priority]** (iv_5) [10].
- ⑥ **Experience Level [EL=1, More experience]** (iv_6): We recorded the experience level of each of the included employees in our retrospective study from their LinkedIn account. The average professional software development experience of participants is 10.5 (range 4 to 25) [9, 32].
- ⑦ **Task Level [TL=1, Sub-task]** (iv_7): the abstraction level of task [31]. We used ParentId column of the dataset to identify task levels.
- ⑧ **Task Stage [TS=1, Late stage]** (iv_8): the completion state of the task [16, 25]. We used temporal task logs and manually analyzed this dataset to identify the completion level of each task.

lower productivity, and developers who are involved in several projects generate more output than others. Similarly, Meyer *et al.* [24] conducted two studies to investigate software developers' personal perception of productivity and the factors which impact this productivity. The results of both studies revealed that developers perceive their day as productive when they complete many or big tasks without interruptions or context switches. However, they observed that participants performed significant task and activity switching while still feeling productive. In a follow-up study, Meyer *et al.* [23] found work habits and perceived productivity are related with each other and identified the time, user input, emails, and planned meetings as factors influencing productivity. Abad *et al.* [1–4] recently conducted four studies to investigate the disruptiveness of task switching in software development projects as well as in requirements engineering tasks. They investigated the impact of interruption length on the duration of interrupted tasks and found that *interruption length* of a specific task, regardless of the type of this task, does not influence its duration significantly. Moreover, they found that, compared to other types of development tasks,

requirements engineering tasks are the most vulnerable tasks to task switching and interruptions.

In terms of the frequency of task switching and developers' productivity, Tregubov *et al.* [35] conducted a retrospective analysis and propose a way to evaluate the number of cross-project interruptions using self-reported develop work logs. The authors reported that developers who, on a typical day, are involved in two or more projects, spend 17% of their development effort on cross-project interruptions. While the results of this work reveal a strong correlation between the number of projects and number of reported interruptions, it shows the correlation between the number of projects and effort spent on cross-project interruptions is relatively weak. Cruz *et al.* [13] conducted a large-scale study to investigate the impact of work fragmentations on developers' productivity and found that work fragmentation is positively correlated with lower observed productivity for an entire session and longer suspension lengths strengthen this effect. Chong and Siino [11] compared the behaviour and the disruptive impact of interruptions among paired and solo programmers. They found that various

interruption characteristics such as time, type, and length of the interruptions as well as strategies for handling work interruptions are significantly different between paired and solo programmers. Similarly, *Ko et al.* [18] conducted a study to understand information needs and the behaviour of task switching and interruptions in collocated software development teams. They found that coworkers are the most frequent source of information in software development teams which causes continual unavoidable task switching and interruptions due to an information need.

Our study confirms some of these results such as the negative impact of task switching on developers' productivity as well as multitasking challenges facing software development teams. Our study extends previous research in the following ways: (1) we model and investigate a comprehensive set of interruption characteristics including task-specific and context-specific factors and study the impact of these factors on task interruptions in various types of software development tasks; (2) we provide a comparison between various development tasks (i.e. programming, testing, architecture design, interface design, and deployment) in terms of their vulnerability to interruptions and task switching. The comprehensiveness of this work in terms of the size of our datasets and the number of dependent and independent variables further builds on these past contributions.

3 METHODS

To achieve our study goals we followed a mixed methods approach including: (1) a longitudinal data analysis on 4,910 recorded tasks of 17 professional software developers, and (2) a user survey with 132 software practitioners to complement the quantitative results with developer perception on task switching and interruptions.

3.1 Study 1: Retrospective Analysis

To gain a broad view of how disruptive task switching and interruptions can be varied by interruption characteristics, we conduct a longitudinal, retrospective study of 4,910 recorded tasks of 17 professional software developers. During the 1.6 years of this study, we developed and tested our conceptual framework (e.g. dependent, independent, and confounding variables) through two exploratory studies. The first study was conducted on 7,770 recorded tasks of 10 employees to ensure dataset quality and to identify potential confounding variables, such as interruption source and type, experience level, and task stage. The second study explores the impact of various interruption characteristics on the disruptiveness of a very specific type of software development tasks and helped to garner additional insights into the problem of task switching in software development teams to better formulate the research's conceptual framework [1, 3, 4]. We conduct this study in collaboration with Arcurve³, a large Calgary independent software services company. The datasets required for these studies were collected from Arcurve's task-based bug tracking and project management tool (i.e. Fogbugz⁴). For each employee, we recorded 100 interruptions giving us 1700 recorded interruptions⁵.

³www.arcurve.com

⁴<http://www.fogcreek.com/fogbugz>

⁵The data extraction form and a sample dataset collected for one employee are available <http://wcm.ualgary.ca/zshakeri/projects>

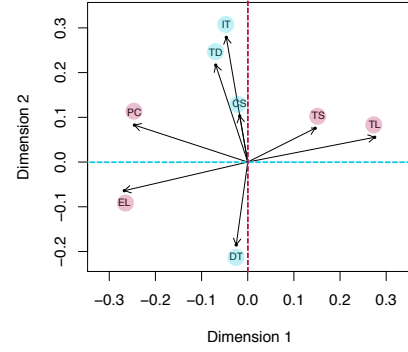


Figure 3: Loading plot for (IV_{1-8}) binary data

3.2 Study 2: User Survey

To garner additional qualitative insights into developers' perception of task switching and interruptions, we use a survey. We sent an online survey to 800 professional software developers working at companies of various sizes (e.g. Microsoft, Tableau Software, Ericsson, Bosch, and Cisco). The survey included 30 questions using multiple choice, Likert scale, and open-ended questions. We asked participants about their job roles, development experience in general, their perception of task switching and productivity and the interruption factors which influence their productivity. We received 132 complete responses (17% response rate). Of all 132 participants, 90 (68%) listed their job as a programmer, 18 (14%) as a software architect, 16 (12%) as a tester, 5 (4%) as project manager and 3 (2%) as requirements engineer. The average professional software development experience per participant was 11.3 years (median: 8; range 3 to 40). The majority (99 or 75%) reported the size of their company (i.e. s = number of employees) $s \geq 1000$, 11 (8%): $100 \leq s < 1000$; 7 (5%): $50 \leq s < 100$, and 8 (6%): $s < 50$. As an incentive, survey respondents were given the option of being entered into a raffle to win one of the \$50US Amazon gift cards.

3.3 Conceptual Framework

The conceptual framework for our study draws from several lines of research and theory including multitasking studies [36], the Memory of Goals [6] and multitasking theories [31], and studies on developers' productivity and task management [23, 24]. Recall Section 2.1 discusses eight independent (IV_{1-8}) and two dependent variables $(\Delta, |w|)$ that are the major constructs of our study. To help interpret the results more easily, we apply *homogeneity analysis* (i.e. Multiple Correspondence Analysis (MCA) [12]) on IV_{1-8} to explore and summarize the underlying variable structure. As we recorded all of our independent variables in binary format, we used the *non-linear Principal Component Analysis (PCA)* approach, a multivariate method for categorical data. To implement this approach, we used the `homals` function of package `homals`⁶ in **R**. The loading plot presented in Figure 3 helps identify variables that most contribute to each dimension. The loading scores of variables in each dimension are used as coordinates. The distance from each point (i.e. variable) to the abscissa (i.e. Dimension 1) or the ordinate (i.e. Dimension 2) gives a measure of the contribution of the point to each dimension. The greater the perpendicular distance from each point to an axis, the stronger the contribution

⁶<https://cran.r-project.org/web/packages/homals/homals.pdf>

of that point to the corresponding dimension [12]. As illustrated in Figure 3, Dimension 1 has high loadings on ν_{1-4} (i.e. CS, TD, IT, and DT) and describes **context-specific characteristics** such as the context, type, and source of the task switching. Likewise, variables ν_{5-8} (i.e. PC, EL, TL, and TS) contribute to Dimension 2, which describes **task-specific characteristics** such as the abstraction level and the priority of the task as well as the required knowledge for performing the task. In the rest of this paper, we use these two dimensions for reporting and interpreting the results.

3.4 Research Questions (RQs)

We formulated the following research questions:

RQ1- Task-specific Vulnerability: How do various interruption characteristics impact the vulnerability of programming, testing, architecture design, UI design, and deployment tasks?

RQ2- Comparative Vulnerability: Which types of development tasks are more vulnerable to task switching/interruptions?

RQ3- Two-way Impact: How does the interaction between various interruption characteristics (ν_{1-8}) influence the vulnerability of development tasks to interruptions?

3.5 Data Analysis

To test for the impact of disruptiveness factors and the difference between various task types (RQs 1-2), we use the non-parametric Kruskal-Wallis and Kruskal-Wallis posthoc tests, respectively. To determine the statistical significance we use the p -values (≤ 0.05), and report as significant, differences at 95% confidence interval, which we use to compare the disruptiveness of interruptions among different task types. Additionally, to check the correlation between participants' responses to survey questions, we use Spearman's rank test and define $|\rho| \geq 0.50$ as a strong correlation coefficient. To model the cross-factor impact of disruptiveness factors (RQ3) we use the Scheirer-Ray-Hare (SRH) test, a non-parametric two-way ANOVA and an extension of the Kruskal-Wallis test. As a high correlation between predictor variables impact the statistical tests of predictors individually, we first applied Phi coefficient tests to statistically test the correlation between all of the independent variables for each of programming, testing, architecture/UI design, and deployment task types. For all correlated factors, we only use the two-way component of SRH tests and to statistically test the significant impact of individual disruptiveness factors on each task type, we applied the Kruskal-Wallis posthoc tests. To analyze the open-ended questions of the survey, we use a modified version of the grounded theory method [33], as a qualitative text analysis method, and use the Saturate App⁷ tool to code the survey responses.

4 RESULTS

Practitioners' Perceptions of Task Switching and Interruptions: When asked about whether participants consider task switching a type of interruption, 107 (81%) stated that they consider task switching a specific type of interruptions because there

Table 1: Top 5 reasons for self-interruptions (%(#)) represents percentage(number) of survey participants)

Reasons for self-interruptions/task switchings	%(#)
Being blocked on a task (e.g. tool obstacles, technical issues)	37 (30%)
Getting sidetracked to other tasks (e.g. remembering other tasks, concentration lapse)	28 (23%)
Planning issues and priority changes (e.g. tasks with near due dates, short term deadlines)	23 (19%)
A need for more information/ technical knowledge (e.g. lack of documentation, waiting for feedback)	20 (16%)
Getting bored with the task (e.g. complex and lengthy tasks)	15 (12%)

Table 2: RQ1- Impact of interruption characteristics on different task types. [●] represents characteristics that make interruptions significantly more disruptive (based on 95% confidence analysis).

	Pairs	Arch		Prog		Test		UI		Dep	
		Δ	$ w $	Δ	$ w $	Δ	$ w $	Δ	$ w $	Δ	$ w $
Context-specific Factors	Kruskal Wallis	0.01	0.06	0.002	0.03	0.3	0.06	0.1	0.6	0.03	0.08
	same project										
	diff project	●		●	●	5e-6	5e-11			●	
	Kruskal Wallis	0.5	0.1	0.06	2e-9			0.8	0.7	0.2	0.01
	diff type										
	same type			●	●	●	●			●	●
	Kruskal Wallis	0.04	0.2	2e-9	2e-8	2e-9	3e-9	0.02	0.5	0.01	0.004
	self-interruption	●		●	●	●	●	●	●	●	●
	ext-interruption	●		●	●	●	●	●	●	●	●
	Kruskal Wallis	0.5	0.1	0.001	1e-5	0.01	0.03	4e-8	0.2	0.6	0.9
Task-specific Factors	morning			●	●	●	●	●	●		
	afternoon			●	●	●	●	●	●		
	Kruskal Wallis	0.2	0.4	0.5	1e-7	0.6	0.01	0.02	0.8	0.9	0.8
	same priority				●	●	●	●	●		
	diff priority										
	Kruskal Wallis	0.2	0.8	0.6	0.2	0.01	0.2	0.2	0.06	0.01	0.03
	less experience									●	●
	more experience									●	●
	Kruskal Wallis	0.002	0.06	0.03	0.01	0.4	0.2	0.01	0.1	0.01	0.03
	sub-task										
Task-specific Factors	main task	●		●	●			●	●	●	●
	Kruskal Wallis	0.7	0.5	0.3	0.3	0.1	0.2	0.1	0.5	0.1	0.04
	early stage										
Task-specific Factors	late stage									●	

is always some ramp-up time when switching between tasks as described by one participant's comment: "Saying that task switching is not an interruption sounds like multitasking is possible. It is not possible and changing the task will interrupt the other task every time and it takes approximately 5-20 minutes to get into the flow state on the task at hand every time there is a switch". We asked survey participants to list the main reasons that would make them have unplanned task switching. We iterated through the responses using the grounded theory approach [33]. Recall from Table 1, getting blocked or getting sidetracked to other tasks, planning issues, a need for more information, and boredom are the most common written responses to this question.

4.1 RQ1- Task-specific Vulnerability

We follow a template and posed 80 null hypotheses to explore factors that may explain the disruptiveness of interruptions in various types of software development tasks: $H_0 = \text{Interruption characteristic } \nu_i \text{ does not impact the } \Delta \text{ and/or } |w| \text{ of task switchings in task } \langle T \rangle$. Where Δ denotes the suspension period, $|w|$ the length of nested task switching, and $\langle T \rangle$ denotes the task type. As illustrated in Table 2, of 34 (43%) rejected tests, 21 (62%) are related to *contextual* factors, and 13 (38%) are related to *task-specific* factors. This implies that, compared to task-specific factors, contextual factors (e.g. context switching and interruption type) are more potent determinants of task switching disruptiveness in software development tasks.

Finding 1-1: The interruption type (i.e. self/external) significantly impacts at least one disruptiveness factor for **all** of the task types under study. As illustrated in Table 2, **self-interruptions**

⁷ www.saturateapp.com/

make task switching and interruptions more disruptive by negatively impacting the length of the suspension period and the number of nested interruptions. Task level (i.e. sub-task/main) comes next, with significant impact on four task types (i.e. architecture, programming, UI, and deployment). Context switching and type switching each negatively impacts three task types.

Finding 1-2: Priority change, daytime, and type difference are characteristics that significantly impact both programming and testing tasks' interruptions. Looking at Table 2, the 95% confidence analysis shows that afternoon interruption or switching to another task with the same priority, or a different type makes programming/testing task interruptions more vulnerable. Moreover, while context switching does not significantly impact the vulnerability of testing and UI tasks to interruptions, switching to a different project negatively impacts the Δ of architectural, programming, and deployment tasks.

Finding 1-3: Following the results of the Kruskal-Wallis tests, only testing interruptions are significantly impacted by the *experience level* ($p=0.01$). Table 2 shows less experienced testers are more vulnerable to interruptions than experienced ones. Likewise, *task stage* impacts only one task type (i.e. deployment tasks).

Discussion 1-1: Although our analysis revealed the statistically significant negative impact of self-interruptions on the vulnerability of *all* development tasks, 107 (81%) participants stated *external-interruptions* are more disruptive than self-interruptions. When asked with an open-ended question about the impact of interruption type on their productivity, most of the participants who selected external interruptions, stated external interruptions are unexpected and are not in their control so are more disruptive. They believed they cannot control the timing of these interruptions which subsequently negatively impacts their performance when they resume the interrupted task, as evidenced in the following quote from one of the participants: *"I tend not to have control over these interruptions and thus I need to follow what they are saying and find a way to make what they are saying happen, and this causes me to become very involved with that one thing which takes time"*. However, the results of two recent studies conducted by Katidioti *et al.* [17] comparing the disruptiveness of self and external interruptions support the results of our quantitative analysis and reveal that external-interruptions are less disruptive than self-interruptions. Similarly, a recent study by Adler and Benbunan-Fich [5] shows that more self-interruptions result in lower accuracy in resumed tasks which causes performance difficulties and consequently sub-optimal results. Another participant of our survey who selected self-interruptions as more disruptive stated that: *"External interruptions are disruptive, but do not necessarily add more items to my cognitive stack. Internal interruptions are always caused by me having (or perceiving myself to have) too many tasks to solve"*.

We speculate that the difference between our survey results and the results of our retrospective analysis and existing theoretical and practical evidence could be due to the high frequency of external interruptions in software development environments. We asked survey participants to, on a scale from 1 to 100, rate what portion of their task switching and interruptions in a day are triggered by an external event. It can be seen from Figure 4a that responses given to this question are slightly skewed to the left which implies that frequencies are more towards the higher side, with mean (and

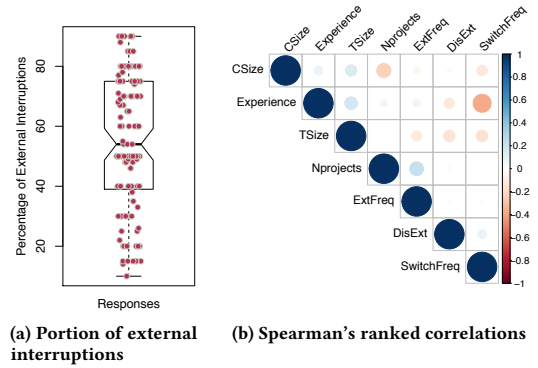


Figure 4: Perceived frequency and disruptiveness of external interruptions and correlation analysis. [CSize=Company size, TSize=Team Size, NProject= # of projects that participants are involved in, ExtFreq= the frequency of external interruptions, DisExt= Disruptiveness of external interruptions, SwitchFreq= the frequency of task switching]

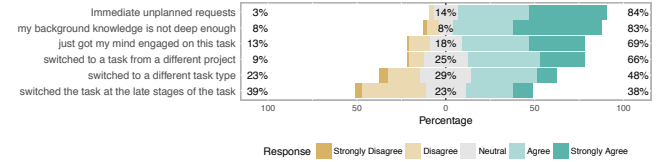


Figure 5: Perceived impact of interruption characteristics

median) values of 54% (range 10-90%). Moreover, we further investigated the association between the disruptiveness and frequency of external interruptions reported by participants and other factors such as their company and team size as well as their experience level and the number of projects they contribute to on a typical day. Spearman's rank correlation tests (summarized in Figure 4b) show the perceived frequency and the disruptiveness of external interruptions do not correlate with their team size, experience level, or the number of projects they are involved in (e.g. TSize-ExtFreq: $\rho = -0.12, p=0.2$; TSize-DisExt: $\rho = -0.15, p=0.12$).

Discussion 1-2: We asked survey respondents to rate the negative impact of context and type switching on a Likert-scale. 120 (91%) and 102 (77%) of the participants indicated neutrality or agreement about the negative impact of context switching and type changes, respectively (Figure 5). The participants predominantly stated that context switching requires a different mindset which places more demands on cognitive resources and makes task switching more disruptive: *"while it depends on how much you have to remember about a specific task/project, context-switching can require more ramp-up because there's more context you have to bring back up"*. This finding is supported by existing literature [23, 24, 35] evaluating the negative impact of context switching on work fragmentation and consequently on developers' productivity and quality of work produced.

Discussion 1-3: While our analysis shows a limited contribution of *experience level* to the vulnerability of development tasks with interruptions, 110 (83%) participants stated that task switching in situations where their background knowledge of performing a task is shallow or they are learning, negatively impacts their performance in the primary task: *"[...] I don't have the most structured*

Table 3: RQ2- Comparison between various development tasks concerning their vulnerability to interruption

Pairs	Context-specific (Dimension 2)								Task-specific (Dimension 1)							
	different context ν_1 (CS=1)		different type ν_2 (TD=1)		interruption type ν_3 (IT=1)		daytime ν_4 (DT=1)		priority change ν_5 (PC=1)		experience level ν_6 (EL=0)		task level ν_7 (TL=1)		temporal stage ν_8 (TS=0)	
	Δ	$ w $	Δ	$ w $	Δ	$ w $	Δ	$ w $	Δ	$ w $	Δ	$ w $	Δ	$ w $	Δ	$ w $
	$3e-4$	$2e-4$	0.001	0.001	0.4	0.05	0.02	0.02	$2e-5$	$4e-6$	$2e-4$	0.003	$4e-4$	$1e-4$	0.1	0.03
Kruskal-Wallis																
Programming-Architecture	0.02*	0.03*	0.2	0.2	0.2	0.04	0.03*	0.1	0.004*	0.003*	0.8	0.04*	0.01	0.05	0.3	0.2
Programming-Test	0.001	0.001	0.01	0.1	0.4	0.9	0.6	0.4	0.003	0.3	0.3	0.4	0.3	0.25	0.07	0.2
Programming-UI	0.5	0.03*	0.2	0.05	0.3	0.001	0.04	0.02	0.4	0.2	0.04	0.01	0.04	0.02	0.1	0.08
Programming-Deployment	0.1	0.06	0.1	0.01	0.3	0.2	0.9	0.05	0.02	0.01	0.01	0.01	0.01*	0.02*	0.1	0.01
Test-Architecture	0.05*	0.03*	0.9	0.8	0.1	0.04	0.07	0.3	0.3	0.6	0.7	0.4	0.05	0.3	0.9	0.6
Test-UI	0.2	0.04*	0.9	0.02*	0.2	0.01	0.5	0.9	0.2	0.1	0.01	0.01	0.2	0.2	0.4	0.2
Test-Deployment	0.2	0.001	0.01	0.002	0.5	0.2	0.02	0.02	0.1	0.03	0.1	0.04	0.001*	0.002*	0.02	0.001
Architecture-UI	0.9	0.9	0.9	0.7	0.9	0.5	0.07	0.3	0.07	0.1	0.1	0.2	0.4	0.8	0.6	0.5
Architecture-Deployment	0.08	0.02	0.04	0.001	0.2	0.02	0.1	0.01	0.4	0.04*	0.1	0.02	0.2	0.02	0.05	0.003
Deployment-UI	0.1	0.06	0.3	0.01	0.2	0.01	0.001	0.002	0.02	0.01	0.001	$1e-4$	0.1	0.6	0.02	0.001

*: The p-value of the alternative value of the corresponding variable.

learning process, so sometimes the structure is not really clear in my head until I have explored a lot of it. If the structure is incomplete, then it's harder to remember, which means that any interruption will have a much worse impact on it than if I already knew the relevant area of code". Researchers have studied the effect of experience level on the cognitive load of tasks. Sweller [34] and Gregory *et al.* [32] argue that experts have the ability to recognize the *problem state* from their previous experiences and accurately recall the information required for resuming their interrupted tasks. Conversely, novices are not able to memorize the problem state of their previous tasks and are forced to use their general problem-solving techniques to resume their interrupted tasks.

Figure 5 shows 91 (69%) participants considered early stage interruptions as a factor that negatively affects their performance after resuming the primary task. The most common written response was that the early investment in a task is critical to building context about an issue and determining next steps when returning from an interruption. This is particularly true in the early stages of a new project because "early stage interruptions result in nearly a perfect storm of wasted time since the time I spent getting engaged had no pay-off". Moreover, only 50 (38%) respondents considered late stage interruptions disruptive: "If the end is in sight, all the necessary work is laid out and is pretty easy to do without much thought. You've likely figured out the main points of the task if you are almost complete, at this point it's a matter of getting the work done and not figuring out how to do it". However, our retrospective analysis revealed that only deployment tasks are impacted by the temporal point of interruptions ($p = 0.04$), and this factor does not significantly impact the vulnerability of other development tasks to interruptions. Contrary to the survey results and the results of our repository analysis, several studies (e.g. [14, 25]) investigated the impact of task stage on the cognitive cost of interruptions and found that middle or late stage interruptions cause longer suspension period (Δ) and consequently decrease in performance and work quality. This difference raises questions about the cognitive cost of interruptions at different stages of a task and implies the need for a further investigation on this factor (i.e. TS).

Practitioner's corner 1: Considering the negative impact of self-interruptions on software developers' productivity (as discussed in Finding₁₋₁ and Discussion₁₋₁), we recommend software developers minimize the frequency of their voluntary task switching. We also recommend that frequent context switching at either task type or project level negatively impacts programmers and testers' productivity by causing fragmented work and longer suspension length. Thus, since switching back and forth between

different projects and task types decreases efficiency by forcing loading and unloading of context per switch, it might be more efficient if developers ask their questions from co-workers working on the same project/task type. Further, as stated by our survey participants, less experienced software developers find it harder to capture the context they were in before switching their primary task and they are most likely to need to backtrack further when they resume their interrupted tasks. Thus, software developers should ask their unplanned questions from co-workers who are more experienced in the topic related to their ongoing task. Consistent with other research [14, 25] and stated by 50 (38%) participants of our survey, switching a task at late stages of the task causes more cognitive cost when recalling the task's context: "I have to rethink from the beginning to make sure that there was no mistake in the previous thoughts". However, as stated by one of the survey participants: "It depends more on complexity at the stage versus which stage in general. I have found it quite easy to resume later stage tasks if they are not complex. A lot of software development tasks are complex though so it could tend to be harder". These apparent conflicts suggest additional research on this factor is required.

4.2 RQ2- Comparative Vulnerability

We posed 160 null hypotheses following this template: $H_0 = \text{The disruptive impact of } \nu_i \text{ on } \Delta \text{ and/or } |w| \text{ is not different between tasks } \langle T, T' \rangle$, where $i \in \{1, 2, \dots, 8\}$ and ν_i , and $\Delta/|w|$ denote independent variables and disruptive factors, respectively. T and T' represent two different task types for all possible pairs of task types (i.e. $\binom{5}{2} = 10$ pairs). Table 3 presents the p-value for each of these tests. The results of our 95% confidence interval analysis (e.g. Figure 6a-q) show that in all cases that task or context-specific factors make a significant difference between deployment and other development tasks, deployment tasks are more vulnerable to interruptions than other task types. This could be because deployment tasks are highly interdependent on different tasks within a development process, which makes their resumption more complicated due to the associated tasks.

Finding 2-1: The results of Kruskal-Wallis tests show that *priority change* makes a statistically significant difference ($p = 0.002$) between the suspension length (Δ) for programming and testing tasks (Table 3). Likewise, *experience level* makes a significant difference between the Δ and the $|w|$ of each of programming and testing tasks, and UI tasks. Regarding the *Task level*, there is a significant difference in Δ and $|w|$ between interrupted low-level programming tasks and each of architecture and UI design tasks. There is also a significant difference between switching low-level

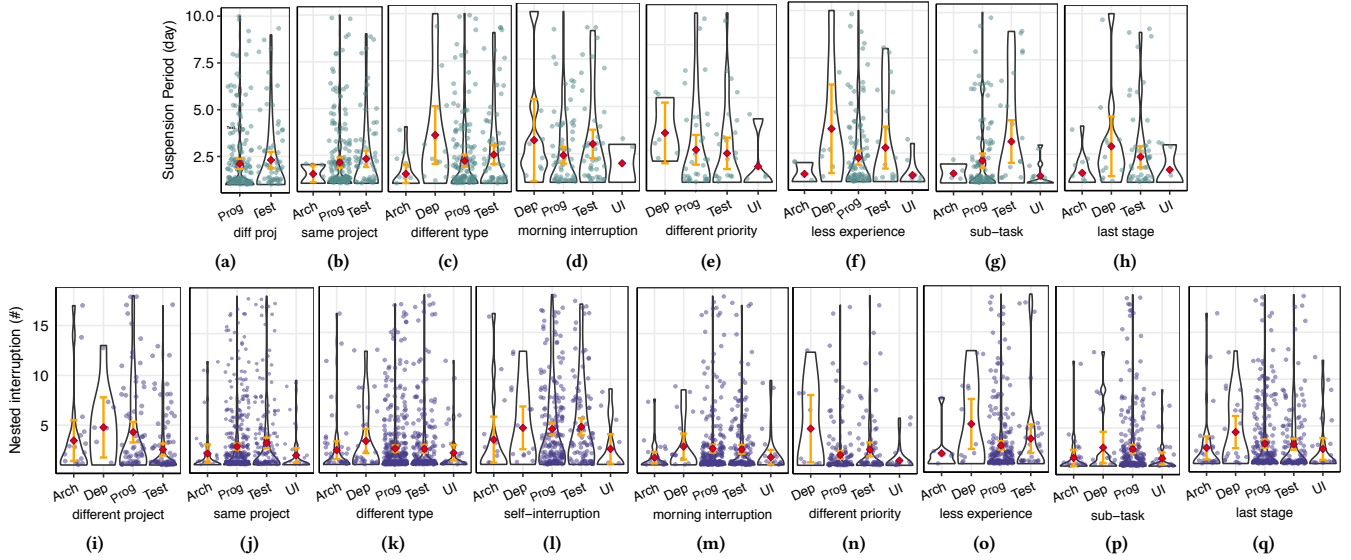


Figure 6: RQ2- 95% confidence interval of sample means for disruptiveness of interruption characteristics in development tasks

testing and low-level architectural tasks with respect to suspension length. Since the Kruskal-Wallis test only identifies that there is a difference, rather than where the differences lie, we used 95% confidence intervals (see Figure 6a-q) to perform the comparative vulnerability analysis. We use comparison patterns to describe our findings in the following.

Finding 2-2: For all interruption characteristics (ν_i) that make a statistically significant difference between tasks $\langle T, T' \rangle$, we provide the following *comparative patterns* for task-specific factors. These patterns compare the vulnerability of two task types $\langle T, T' \rangle$ to interruption using Δ and $|w|$ measures.

- **Priority Change [PC]** (ν_5), (e.g. Figure 6e)
 $\langle PC = 1 \rangle : \Delta_T > \Delta_{T'}$ if $\langle \text{Prog, Test} \rangle$
- **Experience Level [EL]** (ν_6), (e.g. Figure 6f, o)
 $\langle EL = 0 \rangle : \Delta_T > \Delta_{T'}, |w|_T > |w|_{T'}$ if $\langle \{\text{Prog, Test}\}, \text{UI} \rangle$
- **Task Level [TL]** (ν_7), F(e.g. Figure 6g, p)
 $\langle TL = 1 \rangle \begin{cases} \Delta_T > \Delta_{T'}, |w|_T > |w|_{T'} & \text{if } \langle \text{Prog, } \{\text{Arch, UI}\} \rangle \\ \Delta_T > \Delta_{T'} & \text{if } \langle \text{Test, Arch} \rangle \end{cases}$

Finding 2-3: We provide the following *comparative patterns* for context-specific factors.

- **Context Switching [CS]** (ν_1), (e.g. Figure 6a-b, i-j)
 $\langle CS = 1 \rangle : \Delta_T > \Delta_{T'}, |w|_T < |w|_{T'}$ if $\langle \text{Test, Prog} \rangle$
- $\langle CS = 0 \rangle \begin{cases} \Delta_T > \Delta_{T'}, |w|_T > |w|_{T'} & \text{if } \langle \{\text{Prog, Test}\}, \text{Arch} \rangle \\ |w|_T > |w|_{T'} & \text{if } \langle \{\text{Prog, Test}\}, \text{UI} \rangle \end{cases}$
- **Type Difference [TD]** (ν_2), F(e.g. Figure 6c, k)
 $\langle TD = 1 \rangle \begin{cases} \Delta_T > \Delta_{T'} & \text{if } \langle \text{Test, Prog} \rangle \\ |w|_T > |w|_{T'} & \text{if } \langle \{\text{Prog, Test}\}, \text{UI} \rangle \end{cases}$
- **Interruption Type [IT]** (ν_3), (e.g. Figure 6l)

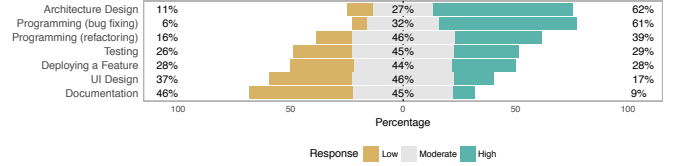


Figure 7: Perceived vulnerability of different development tasks to interruption

$\langle IT = 1 \rangle : |w|_T > |w|_{T'}$ if $\langle \text{Prog, } \{\text{Arch, UI}\} \rangle$
 * The IT factor does not make any significant difference of Δ between different task types.

- **Daytime [DT]** (ν_4), (e.g. Figure 6d, m)
 $\langle DT = 1 \rangle : \Delta_T > \Delta_{T'}, |w|_T > |w|_{T'}$ if $\langle \text{Prog, UI} \rangle$
 $\langle DT = 0 \rangle : \Delta_T > \Delta_{T'}$ if $\langle \text{Prog, Arch} \rangle$

Discussion 2: Based on the results of Findings 2-1 and 2-2, in all cases where there is a significant difference between the vulnerability of programming and testing tasks and other task types ($p < 0.05$), these two types are more vulnerable to task switching and interruption. This finding is consistent with the experimental evidence and theoretical analysis conducted by Sweller [34], which shows that solving problems requiring a large number of items be stored in human short-term memory may contribute to excessive cognitive load. Insofar, as programming and testing tasks require a high number of active statements in developers' *working memory*, which contributes to a higher workload, it is reasonable to expect that switching *programming* and *testing* tasks make them more vulnerable to task switching comparing to architectural and UI tasks. However, when we asked survey respondents about the negative impact of task switching/interruption on different types of development tasks (responses are summarized in Figure 7), 117 (89%) participant reported high or moderate levels of the negative impact of task switching on *architecture design* tasks (i.e. High: 62%, Moderate: 27%). *Programming* and *testing* tasks come next, with

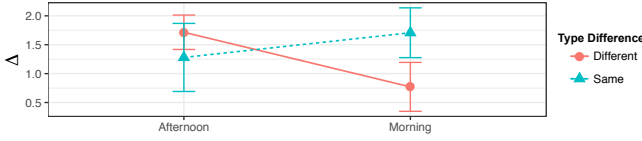


Figure 8: 95% confidence interval of TD/DT factors interaction (Scheirer-Ray-Hare test: $p=0.01$)

each of them being $51(\pm 11)\%$ and 39% level of agreement. However, looking at comparative patterns explored by our retrospective analysis (see Table 3 and Figure 6), we note that *Architectural* tasks in *all* of the cases are significantly different from other task types and are less vulnerable to interruptions. We investigate this difference by conducting a comparison between the survey responses relating to the vulnerability of different development tasks to interruption, grouped by the participants' reported job roles. The responses to the task type associated with each job role received higher rating compared to other task types, showing respondent's job role impacts the responses to this question. Moreover, we studied the association between the perceived vulnerability of each task type and the experience level of respondents. The results of Spearman's rank correlation tests show that the perceived level of vulnerability ranked by developers does not correlate with their experience level (e.g. Test: $\rho=0.13$, $p=0.78$).

Considering the impact of priority change (Finding₂₋₁), switching to a task with a higher priority makes the suspension period for programming tasks significantly longer than testing tasks (i.e. $\Delta_{prog} > \Delta_{test}$, $p=0.002$). Our survey responses also reflect the perceived negative impact of priority change requests on developers' productivity. 111 (84%) participants (strongly) agreed with the disruptiveness of unplanned and immediate interruptions such as priority change requests, as in: "Unplanned requests like high-priority defect fixes don't give me time to save my mental state into the code or the documentation [...] the less likely I can return easily". Conversely, compared to programming tasks, testing tasks are more vulnerable to context and type switching (Figure 6a-c), as stated by one of our survey participants: "As testing can take a different type of mindset than a typical development phase, if switching occurs at mid-task collecting thoughts to return to the task's context can be disruptive and time-consuming".

Practitioner's corner 2: Due to the problem-solving nature of programming and testing tasks, and knowing that human short-term memory is severely limited [6, 34] and cannot accommodate a large number of items, we recommend practitioners minimize switching programming and testing tasks. Further, considering that testing tasks are more vulnerable to context-switching than programming, architecture, and UI design tasks, we propose that it might be more efficient if testers minimize their project switches or they respond to fewer context-switching requests.

4.3 RQ3- Two-way Impact

We consider cross-factor correlations to assess the relationship strength among iv_{1-8} . Since all of the independent variables of our repository analysis are recorded in a binary format, the Phi coefficient test is used to determine the degree and the strength of association between these variables. We then analyze the two-way interaction of these factors on the disruptiveness of interruptions in software development tasks (see Figure 8). The gray-highlighted

cells in Table 4 show the correlation and the interaction between each pair of factors, and the colored circles denote the strength of these correlations.

Finding 3-1: The Phi correlation tests show that for all of the task types studied, there is a significant positive correlation ($\phi > 0.50$, $df=1$, $\chi^2 > 10.8$, $p < 0.001$) between *type difference* and *interruption type* factors. This implies that self-initiated task switchings are mainly associated with a change in the task type. Moreover, in all task types except testing, context switching and experience level variables are negatively correlated with the task level (CS: $\phi \leq -0.64$, $df=1$, $\chi^2 > 10.8$, $p < 0.001$; EL: $\phi \leq -0.80$, $df=1$, $\chi^2 > 10.8$, $p < 0.001$), indicating that for more experienced developers task or context switching are usually high-level tasks.

Finding 3-2: Regarding the interruption timing, there is a significant positive correlation between interruption type and daytime variables for programming, testing, and UI design tasks ($\phi \geq 0.52$, $p < 0.001$). This implies self-initiated interruptions usually happen in the morning. In addition, self-interruptions are associated with interruptions characterized by a priority change ($\phi \leq -0.53$).

Finding 3-3: Table 4 (row TD-DT) shows the interaction between *type change* and *daytime* variables significantly (i.e. SRH tests) impacts both disruptive factors of programming and testing tasks and suspension period of UI task interruptions. For all these three task types, the $\langle 10, 01 \rangle$ (i.e. $\langle$ different type/morning, same type/afternoon $\rangle$) combination negatively impacts the suspension period and for programming and testing tasks the $\langle 00, 11 \rangle$ (i.e. $\langle$ different type/afternoon, same type/morning $\rangle$) negatively impacts the nested interruption parameters.

Finding 3-4: The interaction between task level and type difference variables significantly impacts the disruptiveness of programming and UI interruptions. This interaction is more disruptive when the task switching is characterized as $\langle$ main-task/different type, sub-task/same type $\rangle$.

Finding 3-5: While *experience level* alone does not make any significant difference on the disruptiveness of programming tasks (Tables 2, $p > 0.05$), when it interacts with *type difference*, *interruption type*, or *priority change* these variables significantly impact interruptions to this task type. For example, $\langle 01, 01 \rangle = \langle$ less exp/self-int, more exp/external-int $\rangle$ negatively impact programming task interruptions. Likewise, *context switching* alone does not impact interruptions in testing tasks, but its interaction with *type difference*, *interruption type*, or *priority change* does.

Discussion 3-1: We applied Spearman's rank test on survey responses to questions about the disruptiveness of various interruptions characteristics (see Figure 5). The results reveal that there is a weak correlation between context and type switching variables (i.e. CS/TD: $\rho=0.2$, $p=0.04$). This shows that respondents who rated context switching as a disruptive interruption factor, did so for the type switching factor: "Changing a task type is disruptive if it made me change environment e.g. Launch different servers". Spearman's rank tests also show a weak correlation between type difference (TD) and each of interruption type (IT) and task stage (TS) factors (TD/TS: $\rho=0.19$, $p=0.04$; TD/IT: $\rho=0.22$, $p=0.02$), as in: "the disruptiveness of type switching depends on if I reached a good stopping point before the switch or not [...]". We found there is a correlation between participants' rating to the disruptiveness of context switching (CS) and interruption type (IT) factors (CS/IT:

Table 4: RQ3- Two-way factorial Scheirer-Ray-Hare Test. [CS]: Context Switching, [TD]: Type Difference, [IT]: Interruption Type, [DT]: Daytime, [PC]: Priority Change, [EL]: Experience Level, [TL]: Task Level, [TS]: Task Stage.

Pairs	Architecture			Programming			Test			UI			Deployment		
	ϕ	Δ	$ w $	ϕ	Δ	$ w $	ϕ	Δ	$ w $	ϕ	Δ	$ w $	ϕ	Δ	$ w $
CS-EL	0.42			0.79			0.001			-0.90			0.80		
CS-TL	-0.64			-0.80			-0.13			-0.91			-0.90		
CS-TD	0.11			0.02			-0.33			0.62			0.84		
CS-IT	0.05			0.10			0.21			-0.10			0.56		
CS-PC	-0.76			0.36			-0.46			0.24			0.48		
CS-DT	-0.72			-0.10			0.14			-0.49			-0.31		
CS-TS	-0.42			-0.44			0.36			0.12			-0.83		
EL-TL	-0.80			-0.98			-0.10			-0.99			-0.84		
EL-TD	-0.78			-0.47			-0.38			0.28			0.59		
EL-IT	-0.48			0.63			-0.53			-0.39			0.36		
EL-PC	-0.30			0.77			-0.11			0.59			0.25		
EL-DT	-0.59			-0.27			-0.70			-0.63			-0.42		
EL-TS	-0.50			-0.25			0.62			0.36			-0.78		
TL-TD	0.47			0.39			-0.23			-0.28			-0.63		
TL-IT	0.55			0.23			-0.22			0.40			-0.43		
TL-PC	0.28			-0.73			-0.29			-0.60			-0.39		
TL-DT	0.51			0.15			-0.10			0.66			0.22		
TL-TS	0.32			0.31			0.33			-0.41			0.63		
TD-IT	0.54			0.82			0.74			0.73			0.89		
TD-PC	0.05			-0.69			-0.10			-0.60			0.61		
TD-DT	-0.01			0.36			0.14			0.28			-0.24		
TD-TS	0.27			-0.20			-0.55			-0.64			-0.91		
IT-PC	-0.21			-0.76			-0.53			-0.94			0.61		
IT-DT	0.10			0.63			0.52			0.83			-0.15		
IT-TS	-0.31			-0.41			-0.36			-0.86			-0.73		
PC-DT	0.50			-0.62			-0.16			-0.78			-0.63		
PC-TS	0.35			0.10			-0.47			0.85			-0.59		
DT-TS	0.36			-0.41			-0.54			-0.68			0.52		

• : $0.50 \leq \rho < 0.65$, • : $0.65 \leq \rho < 0.80$, • : $\rho \geq 0.80$,
 • : $-0.65 < \rho \leq -0.50$, • : $-0.80 < \rho \leq -0.65$, • : $\rho \leq -0.80$
 : No interaction

$\rho=0.37$, $p=1e-7$). Similar to the results of our retrospective interaction analysis, respondents who rated context switching as a disruptive factor found external interruptions more disruptive than self-interruptions: “typically the interruptions that come from others are longer reaching - often it means that my skills are needed elsewhere, and so I need to switch tasks or projects for a more extended period, which adds more items to my cognitive stack”.

Discussion 3-2: We propose a set of correlation and interaction patterns that can be used to interpret developers’ task switching behaviour and to investigate the cross-factor impact of task switching characteristics. We present these patterns as:

Correlation Patterns: $\langle T, (iv_i, iv_j), \bullet \rangle$, where i and $j \in \{1, 2, \dots, 8\}$ and denote two distinct interruption characteristics and the color of $\bullet$ presents the direction and the strength of the association between these characteristics. For instance, $\langle Programming, (CS, TL), \bullet \rangle$ indicates there is a strong negative association between the context switching and task level variables in programming tasks’ interruptions.

Interaction Patterns: $\langle (iv_i, iv_j), (\alpha\beta, \bar{\alpha}\bar{\beta}), \Delta/|w|_T \rangle$, which implies the interaction between two distinct interruption characteristics iv_i and iv_j with the values of α and β (i.e. $\alpha, \beta \in \{0, 1\}$) negatively impacts Δ and/or $|w|$ of task T ’s interruptions. For instance, $\langle (TD, PC), (11, 00), \Delta_{Test} \rangle$ indicates that (diff type/same priority, same type/diff priority) significantly impact interruptions of testing tasks and negatively impact their suspension period.

These patterns along with the detailed information presented in Table 4, can be used to guide decision-making and forecasting the consequences of task switching decisions.

Practitioner’s corner 3: While there are various combinations of factors which can impact the disruptiveness of interruptions in

a negative way, the results of this section do not exactly prove that interruptions are always disruptive. There are circumstances where task switching or interruptions can boost developers’ productivity, as stated by one of our survey respondents: “Learning takes time. Sometimes I learn basics for a task then I leave it for the next day which makes me mentally prepared for the task. Or, if a team member asks me a question about a portion of a feature which they are working on, that often gives me clarity about what I am working one”. We propose that task switching is a skill and not an obstacle to work. Designing the development processes in a way to be resilient to interruptions can mitigate the risk of unplanned and disruptive interruptions. For instance, having frequent, small commits help a team keep the amount of work that they have not yet submitted always very small. Mapping each commit to one discrete change to the source code (e.g. refactoring, a failing test, or a TDD cycle) and encoding all of developers’ knowledge about the code into the code itself (e.g. by extracting methods and renaming methods and variables to reflect their meaning) help reduce the cognitive cost of unavoidable task switching and interruptions occur to programming tasks.

5 THREATS TO VALIDITY

Although our longitudinal study used data collected from a single company we argue that our findings generalize. We tried to mitigate this risk by implementing our repository study on a fairly large dataset including various projects from different business domains and employees from different levels of experience. Our data collection and preparation pose another threat to the validity of our results because identifying the interruption type (i.e. self and external) and temporal stage of tasks (i.e. early and late) is not straightforward. The pilot studies we conducted before our main data collection phase helped address this risk. Additionally, the

retrospective dataset associated with each employee was reviewed by at least two hired RA's and the first author of the paper. To evaluate the reliability of our decisions for independent variables that have been recorded manually, we used the Cohen's Kappa statistic, which calculates the degree of agreement between two evaluators. The calculated Kappa value was 0.87, which shows significant agreement according to Landis and Koch [19]. In regard to survey results, we pilot tested the survey questions with three software developers to mitigate the risk of misunderstanding questions. However, the questions still require participant interpretation. We mitigate this risk by adding a comment space for each question and asked respondents to clarify their response or discuss other aspects of the question if they desired. The survey population could be biased towards a specific population so the generalizability of our survey results may have intrinsic limits. We mitigate this by distributing our survey to a large number of potential respondents with different levels of software development experience and from various countries (e.g. Germany, Netherlands, Sweden, Hungary, USA, New Zealand, and Canada).

6 CONCLUSION AND IMPLICATIONS

Interruption, as a form of task switching or sequential multitasking, is an inherent part of software development tasks. Not all of the interruptions should be counted as waste because in some specific cases task switching is unavoidable and can actually increase developers' productivity. Using a mixed-methods study including a retrospective analysis and a survey, we studied the disruptive impact of various interruption characteristics on development tasks interruptions. We found that the problem-solving nature of programming and testing tasks make them more vulnerable to interruptions compared to architecture and UI design tasks. Interestingly, we found self-interruptions negatively impact the disruptiveness of interruptions in all types of development tasks. However, the survey responses reveal that developers seem to believe external-interruptions are more vulnerable than self-interruptions. We also provided a set of recommendations (see *practitioners' corners*) for project managers and practitioners which can be used as a mean to guide decision-making and forecasting the consequences of task switching decisions in software development teams.

We suggest that research in multitasking and task interruptions in the area of software engineering focus on measuring and characterizing the cost of task switching and interruptions. As the differences between our repository analysis and survey data reveal and as supported by recent practical studies (see [23, 24, 36]), the disruptiveness of task switching is most likely to be affected by the context in which the switching occurs. As one of the respondents said: "[...] If someone is working on the same project as I am and we can exchange ideas, that can be a productive task switching. It's also productive for more fire-drill type situations, like fast bug triage."

REFERENCES

- [1] Zahra Shakeri Hossein Abad, Guenther Ruhe, and Mike Bauer. 2017. Task Interruptions in Requirements Engineering: Reality versus Perceptions!. In *Requirements Engineering Conference (RE), 2017 IEEE 25th International*. IEEE, 6–15.
- [2] Zahra Shakeri Hossein Abad, Mohammad Noeen, Didar Zowghi, Behrouz H. Far, and Ken Barker. 2018. Two Sides of the Same Coin: Software Developers' Perceptions of Task Switching and Task Interruption. In *Proceedings of the 22nd International Conference on Evaluation and Assessment in Software Engineering (EASE'18)*. ACM.
- [3] Zahra Shakeri Hossein Abad, Guenther Ruhe, and Mike Bauer. 2017. Understanding Task Interruptions in Service Oriented Software Development Projects: An Exploratory Study. In *Proceedings of the 4th International Workshop on Software Engineering Research and Industrial Practice (SER&IP '17)*. IEEE Press, 34–40.
- [4] Zahra Shakeri Hossein Abad, Alex Shymka, Jenny Le, Noor Hammad, and Guenther Ruhe. 2017. A Visual Narrative Path from Switching to Resuming a Requirements Engineering Task. In *Requirements Engineering Conference (RE), 2017 IEEE 25th International*. IEEE, 442–447.
- [5] Rachel F. Adler and Raquel Benbunan-Fich. 2013. Self-interruptions in Discretionary Multitasking. *Computers in Human Behavior* 29, 4 (2013), 1441–1449.
- [6] Erik M Altmann and J Gregory Trafton. 2002. Memory for Goals: An Activation-based Model. *Cognitive science* 26, 1 (2002), 39–83.
- [7] John R Anderson. 1990. *Cognitive Psychology and Its Implications*. WH Freeman/Times Books/Henry Holt & Co.
- [8] John R Anderson and Christian J Lebiere. 2014. *The Atomic Components of Thought*. Psychology Press.
- [9] Jelmer P Borst, Niels A Taatgen, and Hedderik van Rijn. 2010. The Problem State: A Cognitive Bottleneck in Multitasking. *Journal of Experimental Psychology: Learning, Memory, and Cognition* 36, 2 (2010), 363.
- [10] Jelmer P. Borst, Niels A. Taatgen, and Hedderik van Rijn. 2015. What Makes Interruptions Disruptive?: A Process-Model Account of the Effects of the Problem State Bottleneck on Task Interruption and Resumption. In *Proceedings of the 33rd Annual ACM Conference on Human Factors in Computing Systems (CHI '15)*. ACM, 113–120.
- [11] Jan Chong and Rosanne Siino. 2006. Interruptions on software teams: a comparison of paired and solo programmers. In *Proceedings of the 2006 20th anniversary conference on Computer supported cooperative work*. ACM, 29–38.
- [12] Sten Erik Clausen. 1998. *Applied Correspondence Analysis: An Introduction*. Vol. 121. Sage.
- [13] Luis C Cruz, Heider Sanchez, Victor M González, and Romain Robbes. 2017. Work fragmentation in developer interaction data. *Journal of Software: Evolution and Process* 29, 3 (2017).
- [14] Mary Czerwinski, Edward Cutrell, and Eric Horvitz. 2000. Instant messaging: Effects of relevance and timing. In *People and computers XIV: Proceedings of HCI*, Vol. 2. 71–76.
- [15] Rico Fischer and Franziska Plessow. 2015. Efficient multitasking: parallel versus serial processing of multiple tasks. *Frontiers in psychology* 6 (2015).
- [16] Victor M. González and Gloria Mark. 2004. Constant, Constant, Multi-tasking Crazyiness: Managing Multiple Working Spheres. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '04)*. ACM, 113–120.
- [17] Ioanna Katidioti, Jelmer P. Borst, Marieke K. van Vugt, and Niels A. Taatgen. 2016. Interrupt me: External Interruptions are Less Disruptive Than Self-interruptions. *Computers in Human Behavior* 63, Supplement C (2016), 906–915.
- [18] Andrew J Ko, Robert DeLine, and Gina Venolia. 2007. Information needs in collocated software development teams. In *Software Engineering, 2007. ICSE 2007. 29th International Conference on*. IEEE, 344–353.
- [19] J Richard Landis and Gary G Koch. 1977. The Measurement of Observer Agreement for Categorical Data. *biometrics* (1977), 159–174.
- [20] Gloria Mark, Victor M. Gonzalez, and Justin Harris. 2005. No Task Left Behind?: Examining the Nature of Fragmented Work. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '05)*. ACM, 321–330.
- [21] Daniel C. McFarlane and Kara A. Latorella. 2002. The Scope and Importance of Human Interruption in Human-computer Interaction Design. *Hum.-Comput. Interact.* 17, 1 (2002), 1–61.
- [22] A. N. Meyer, L. E. Barton, G. C. Murphy, T. Zimmermann, and T. Fritz. 2017. The Work Life of Developers: Activities, Switches and Perceived Productivity. *IEEE Transactions on Software Engineering* PP, 99 (2017), 1–1.
- [23] André N Meyer, Laura E Barton, Gail C Murphy, Thomas Zimmermann, and Thomas Fritz. 2017. The Work Life of Developers: Activities, Switches and Perceived Productivity. *IEEE Transactions on Software Engineering* (2017).
- [24] André N. Meyer, Thomas Fritz, Gail C. Murphy, and Thomas Zimmermann. 2014. Software Developers' Perceptions of Productivity. In *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE 2014)*. ACM, 19–29.
- [25] Christopher A Monk, Deborah A Boehm-Davis, and J Gregory Trafton. 2002. The Attentional Costs of Interrupting Task Performance at Various Stages. In *Proceedings of the human factors and ergonomics society annual meeting*, Vol. 46. SAGE Publications Sage CA: Los Angeles, CA, 1824–1828.
- [26] Michael Boyer O'leary, Mark Mortensen, and Anita Williams Woolley. 2011. Multiple team membership: A theoretical model of its effects on productivity and learning for individuals and teams. *Academy of Management Review* 36, 3 (2011), 461–478.
- [27] Chris Parnin. 2010. A Cognitive Neuroscience Perspective on Memory for Programming Tasks. In *In the Proceedings of the 22nd Annual Meeting of the Psychology of Programming Interest Group (PPIG)*.
- [28] Chris Parnin and Spencer Rugaber. 2011. Resumption Strategies for Interrupted Programming Tasks. *Software Quality Journal* 19, 1 (2011), 5–34.
- [29] John Rieman. 1993. The Diary Study: A Workplace-oriented Research Tool to Guide Laboratory Efforts. In *Proceedings of the INTERACT '93 and CHI '93 Conference on Human Factors in Computing Systems (CHI '93)*. ACM, 321–326.
- [30] Dario D Salvucci and Niels A Taatgen. 2010. *The Multitasking Mind*. Oxford University Press.
- [31] Dario D. Salvucci, Niels A. Taatgen, and Jelmer P. Borst. 2009. Toward a Unified Theory of the Multitasking Continuum: From Concurrent Performance to Task Switching, Interruption, and Resumption. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (CHI '09)*. ACM, 1819–1828.
- [32] Gregory Schraw, Michael E Dunkle, and Lisa D Bendixen. 1995. Cognitive Processes in Well-defined and Ill-defined Problem Solving. *Applied Cognitive Psychology* 9, 6 (1995), 523–538.
- [33] K. J. Stol, P. Ralph, and B. Fitzgerald. 2016. Grounded Theory in Software Engineering Research: A Critical Review and Guidelines. In *2016 IEEE/ACM 38th International Conference on Software Engineering (ICSE)*. 120–131.
- [34] John Sweller. 1988. Cognitive Load During Problem Solving: Effects on Learning. *Cognitive science* 12, 2 (1988), 257–285.
- [35] Alexey Tregubov, Barry Boehm, Natalia Rodchenko, and Jo Ann Lane. 2017. Impact of Task Switching and Work Interruptions on Software Development Processes. In *Proceedings of the 2017 International Conference on Software and System Process (ICSSP 2017)*. ACM, 134–138.
- [36] B. Vasilescu, K. Blincoe, Q. Xuan, C. Casalnuovo, D. Damian, P. Devanbu, and V. Filkov. 2016. The Sky Is Not the Limit: Multitasking Across GitHub Projects. In *2016 IEEE/ACM 38th International Conference on Software Engineering (ICSE)*. 994–1005.